UNIT IV: Inheritance & Pointers

Question Bank: Debugging & Pseudo-Code Formulation (Questions Only)

---

1. Syllabus & Topic Scope

- **Inheritance Fundamentals**: Concept of inheritance, code reusability, base class vs. derived class, access visibility specifiers (`public`, `protected`, `private` derivation).
- **Types of Inheritance**: Single, Multilevel, Multiple, Hierarchical, and Hybrid inheritance.
- **Virtual Base Classes**: The Diamond Problem (multipath inheritance duplication and ambiguity), resolving redundant subobjects using the `virtual` base inheritance specifier.
- **Abstract Classes**: Pure virtual functions as interfaces, incomplete classes, instantiability rules.
- **Constructors in Derived Classes**: Constructor execution order (Virtual base classes first, non-virtual base classes in order of declaration, member/composite objects, derived class constructor), passing arguments up the inheritance chain via the member initializer list.
- **Member Classes (Containment / Composition)**: "Has-a" relationship vs. "Is-a" relationship, member object initialization.
- **Pointers & Pointer Operations**: Pointer declaration, address-of operator (`&`), dereference operator (`*`), pointer arithmetic.
- **Pointers to Objects**: Dynamic object instantiation (`new`), arrow operator (`->`), arrays of pointers to objects.
- **The `this` Pointer**: Implicit `this` pointer in non-static member functions, resolving parameter-field identifier shadowing, enabling method chaining (fluent interface) via `return *this;`.
- **Pointers to Derived Classes**: Upcasting (Base class pointer pointing to Derived class object), compile-time early binding limitations, object slicing phenomenon when copying polymorphic objects by value.

---

2. Assessment Instructions & Rules ("How to Do & What to Do")

A. Instructions for Debugging Questions

When debugging inheritance hierarchies and pointer mechanics in C++:

1. Analyze Multipath Inheritance (The Diamond Problem):

- When two intermediate classes inherit from the same base class and are joined by a derived class, check whether the base class was inherited as `virtual`. Missing `virtual` results in duplicate subobjects and member ambiguity errors (`error: request for member is ambiguous`).

2. Verify Constructor Invocation Chains:

- If a base class lacks a parameterless default constructor, verify that every derived class constructor explicitly invokes the base constructor in its **member initializer list**.
- Note that for virtual base classes, the **most derived class** is solely responsible for initializing the virtual base subobject.

3. Inspect Pointer Scope & Object Slicing:

- When using a base class pointer (`Base*`) pointing to a derived object, remember that the compiler only recognizes members declared in `Base`. Calling derived-specific members through `Base*` requires downcasting.
- If a derived object is assigned by value to a base object (`Base b = derivedObj;`), identify the **Object Slicing** bug where derived attributes and polymorphism are stripped.

4. Inspect `this` Pointer Chaining Semantics:

- For method chaining (`obj.setA().setB()`), check whether the method returns a reference `ClassName&` with `return *this;`. Returning a pointer `ClassName*` with `return this;` causes member dot operator syntax to fail.

5. Verify Access Visibility in Inheritance:

- Check whether inheritance is specified as `private` or `protected`. In private inheritance, public base members become private in the derived class, and upcasting outside the class becomes illegal (`inaccessible base`).

6. Required Response Format:

- State exact line numbers and defect category.
- Provide the theoretical explanation referencing class layouts and pointer type checking.
- Provide the clean, compilable corrected code snippet.
- Provide the dry run and console output.

B. Instructions for Pseudo-Code Writing Questions

1. Inheritance & Hierarchy Modeling:

- Clearly state inheritance syntax: `CLASS Derived EXTENDS Base` or `CLASS Derived INHERITS Base1, VIRTUAL Base2`.
- For abstract classes, prefix with `ABSTRACT CLASS` and tag unimplemented methods with `ABSTRACT METHOD`.
- Explicitly depict constructor delegation: `CONSTRUCTOR(...) : BASE(...)`.

2. Pointers & References:

- Use explicit pointer notation: `DECLARE ptr AS POINTER TO BaseType`.
- Dynamic instantiation: `ptr <- NEW DerivedType(...)`.
- Method chaining: `RETURN REF THIS`.

---

3. Section A: Debugging Questions (5 Problems)

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

Topic: Hybrid Inheritance, Virtual Base Classes, Ambiguity

Problem Statement

A university information system models a teaching assistant who is simultaneously a `Student` and an `Employee`, both of which inherit from a common `Person` base class. When attempting to access the `idNumber` field on a `TeachingAssistant` instance, the compiler halts with an ambiguity error.

Buggy Code

```
#include <iostream>
#include <string>

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << std::endl;
    }
};

// Line 12: Standard non-virtual inheritance
class Student : public Person {
public:
    Student(int id) : Person(id) {}
};

// Line 18: Standard non-virtual inheritance
class Employee : public Person {
public:
    Employee(int id) : Person(id) {}
};

// Hybrid inheritance combining Student and Employee
class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int sId, int eId)
        : Student(sId), Employee(eId) {}
};

int main() {
    TeachingAssistant ta(101, 202);

    // Line 33: Ambiguous member access!
    std::cout << "TA ID: " << ta.idNumber << std::endl;

    return 0;
}
```

Student Tasks

1. State the exact compilation error that occurs on **Line 33**.

2. Draw or explain the memory layout of `TeachingAssistant` under non-virtual inheritance. Why are there two separate `idNumber` fields?

3. Refactor the classes using **Virtual Base Class Inheritance** to ensure that only a single, shared instance of `Person` exists.

4. Explain how constructor delegation changes when virtual base classes are used: Which constructor is responsible for invoking `Person(int)`?

Question 2: Missing Base Constructor in Derived Initializer List

Topic: Constructors in Derived Classes, Order of Constructor Execution

Problem Statement

A base class `Device` requires a hardware MAC address during initialization and provides no default constructor. The derived class `Sensor` fails to compile because it attempts to set the base class attribute inside its constructor body instead of using a member initializer list.

Buggy Code

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;

public:
    // Parameterized constructor ONLY - no default constructor provided!
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: " << macAddress << std::endl;
    }
};

class Sensor : public Device {
private:
    int samplingRate;

public:
    // Line 18: Derived constructor fails to explicitly call Device(mac)
    Sensor(std::string mac, int rate) {
        // Attempting to assign in body after base constructor already attempted
        macAddress = mac;
        samplingRate = rate;
    }

    void display() const {
        std::cout << "MAC: " << macAddress << ", Rate: " << samplingRate << " Hz" <<
std::endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Student Tasks

1. Explain why the compiler generates an error before the constructor body of `Sensor` ever executes. What constructor does the compiler attempt to call implicitly?

2. Detail the exact chronological sequence of constructor execution when a derived class object is instantiated.

3. Rewrite the `Sensor` constructor using a member initializer list to delegate arguments to `Device`.

4. Provide the verified terminal output.

Question 3: Base Class Pointer Member Access & Object Slicing

Topic: Pointers to Derived Classes, Static Type Checking, Object Slicing

Problem Statement

A developer uses a base class pointer `Vehicle*` to manage a heap-allocated `SportsCar` object, but encounters a compilation failure when attempting to invoke a derived-specific method. Later, when assigning a `SportsCar` object to a value variable of type `Vehicle`, unexpected object slicing occurs.

Buggy Code

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    void honk() { std::cout << "General Horn" << std::endl; }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo) : Vehicle(s), turboEnabled(turbo) {}
    void engageTurbo() { std::cout << "Turbo Engaged!" << std::endl; }
};

int main() {
    // Upcasting: Base pointer pointing to derived object
    Vehicle *vPtr = new SportsCar(300, true);

    vPtr->honk();

    // Line 22: Attempting to call derived method through Base pointer
    vPtr->engageTurbo(); // Compilation Error!

    // Line 26: Value assignment between polymorphic types
    SportsCar car(250, true);
    Vehicle plainVehicle = car; // Object Slicing occurs here!

    delete vPtr;
    return 0;
}
```

Student Tasks

1. Explain why `vPtr->engageTurbo()` on **Line 22** causes a compilation error even though `vPtr` points to a `SportsCar` in heap memory. How does C++ static type checking govern pointer member access?

2. Define the phenomenon of **Object Slicing** on **Line 26**. What happens to `turboEnabled` during this assignment?

3. Show how to correctly invoke `engageTurbo()` using an explicit downcast, and explain how to prevent slicing by using references or pointers.

4. Provide the corrected, compilable code snippet.

Question 4: Incorrect Return Type in `this` Pointer Method Chaining

Topic: The `this` Pointer, Method Chaining (Fluent Interface)

Problem Statement

A GUI styling builder class `WindowBox` uses the `this` pointer to support method chaining (fluent interface). However, one method returns a pointer `WindowBox*` while another returns a reference `WindowBox&`. Chaining these calls results in syntax errors with the member selection operator `.`.

Buggy Code

```
#include <iostream>

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    // Line 11: Returns pointer 'WindowBox*' instead of reference 'WindowBox&'
    WindowBox* setWidth(int width) {
        this->width = width;
        return this; // Returns pointer!
    }

    // Line 18: Returns reference 'WindowBox&'
    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width << " x " << height << std::endl;
    }
};

int main() {
    WindowBox box;

    // Line 30: Chaining fails compilation
    box.setWidth(800).setHeight(600);

    box.display();
    return 0;
}
```

Student Tasks

1. Analyze the expression `box.setWidth(800).setHeight(600)` on **Line 30**. Why does the compiler reject `.setHeight(600)` after `setWidth(800)` has executed?
2. Differentiate between `this` and `*this` in terms of data type and memory semantics.
3. Correct the return type and return statement of `setWidth` to enable smooth method chaining.
4. Show the verified console output.

Question 5: Private Inheritance Access Violations & Inaccessible Base Cast

Topic: Access Specifiers in Inheritance (`private` derivation), Upcasting

Problem Statement

A secure networking module attempts to reuse a standard networking socket class via `private` inheritance. However, client code in `main()` encounters compilation errors when attempting to call base class methods or upcast the derived object pointer to a base pointer.

Buggy Code

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened." << std::endl;
    }
    void closePort() {
        std::cout << "Port closed." << std::endl;
    }
};

// Line 13: Private inheritance used
class SecureChannel : private RawSocket {
public:
    void establishEncryptedTunnel(int port) {
        openPort(port); // Legal inside derived class
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);

    // Line 24: Calling base method from outside
    channel.closePort(); // Compilation Error!

    // Line 27: Upcasting with private inheritance
    RawSocket *socketPtr = &channel; // Compilation Error!

    return 0;
}
```

Student Tasks

1. Explain what happens to the accessibility of `openPort()` and `closePort()` when `SecureChannel` inherits `RawSocket` privately. Why is `channel.closePort()` on **Line 24** forbidden in `main()`?

2. Explain why `RawSocket *socketPtr = &channel;` on **Line 27** fails compilation with `error: 'RawSocket' is an inaccessible base of 'SecureChannel'`. Under what conditions does C++ allow upcasting?

3. Provide two distinct ways to make `closePort()` accessible:

- Approach A: Using the `using RawSocket::closePort;` access declaration.
- Approach B: Using public inheritance or composition.

4. Write the corrected code using Approach A.

---

4. Section B: Pseudo-Code Writing Questions (5 Problems)

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Topic: Hybrid Inheritance, Virtual Base Classes, Diamond Resolution

Scenario

Design the pseudo-code for a university ERP record management system using **Hybrid Inheritance with Virtual Base Classes**:

- Root Base Class: `Person` (contains `personID`, `fullName`, and `email`).
- Derived Class 1: `Student` virtually inheriting `Person` (adds `gpa`, `creditsEarned`).
- Derived Class 2: `Faculty` virtually inheriting `Person` (adds `academicDepartment`, `rankTitle`).
- Most Derived Class: `TeachingAssistant` inheriting from both `Student` and `Faculty` (adds `stipendAmount`, `assignedCourse`).

Functional Requirements

1. Virtual Base Class Mechanics:

- Prevent duplicate `Person` subobjects in `TeachingAssistant`.
- The `TeachingAssistant` constructor must directly delegate initialization to the virtual `Person` root constructor.

2. Display Method:

- Implement `DisplayTARecord()` that accesses `personID`, `fullName`, `gpa`, `department`, and `stipendAmount` unambiguously.

Student Tasks

1. Write the formal structured pseudo-code showing class definitions, virtual inheritance declarations, and constructor delegation.

2. Trace the instantiation of a `TeachingAssistant` record with sample data and show the output of `DisplayTARecord()`.

Question 2: Polymorphic Payroll with Abstract Base Class & Base Pointers

Topic: Abstract Classes, Pure Virtual Functions, Pointers to Derived Classes

Scenario

Design an enterprise payroll calculation system using an **Abstract Base Class** `Employee` and an array of base class pointers (`Employee*`) to compute salaries polymorphically.

Functional Requirements

1. Abstract Class `Employee`:

- Protected Attributes: `employeeID` (STRING), `name` (STRING).
- Methods:
  - Parameterized constructor.
  - `ABSTRACT METHOD CalculatePay() RETURNS REAL` (Pure virtual function).
  - `DisplayHeader()`: Outputs ID and name.

2. Concrete Subclasses:

- `SalariedEmployee`: Stores `monthlySalary`. Overrides `CalculatePay()` to return `monthlySalary`.
- `HourlyEmployee`: Stores `hourlyWage` and `hoursWorked`. Overrides `CalculatePay()` to compute regular pay for ≤ 40 hours, and $1.5\times$ overtime pay for hours above 40.

3. Batch Processor (`Main`):

- Dynamically allocate an array of base pointers `Employee*`.
- Instantiate one `SalariedEmployee` and one `HourlyEmployee`.
- Iterate through the pointer array, invoke `CalculatePay()` polymorphically via `->`, accumulate total disbursements, and deallocate memory.

Student Tasks

1. Write the structured pseudo-code for the abstract class, derived classes, and batch driver.

2. Construct a dry-run trace table showing pay calculations for:

- Salaried employee: \$8,500/month.
- Hourly employee: \$50/hour for 48 hours worked (40 regular + 8 overtime).

Question 3: Composite Canvas Engine with Member Classes & `this` Chaining

Topic: Member Classes (Composition), `this` Pointer Chaining

Scenario

Design a graphical geometry rendering engine using **Member Classes (Composition / "has-a")** and **Method Chaining via `this`**:

- Sub-object Class: `Point2D` (encapsulates coordinates x, y).
- Container Class: `Circle` contains an instance of `Point2D` as its center (`centerPoint`) and a `radius` scalar.
- The `Circle` methods `SetCenter(x, y)` and `SetRadius(r)` must return a reference to `*this` allowing fluent method chaining (`circle.SetCenter(10, 20).SetRadius(5).Draw()`).

Functional Requirements

1. Class `Point2D`: Fields `x, y` (REAL), parameterized constructor with defaults $(0.0, 0.0)$.

2. Class `Circle`:

- Attributes: `centerPoint` (Point2D), `radius` (REAL).
- Constructor initializes `centerPoint` to origin and `radius` to 1.0.
- Method `SetCenter(newX, newY) RETURNS REF Circle`: Updates coordinates and returns `REF THIS`.
- Method `SetRadius(r) RETURNS REF Circle`: Validates $r > 0$, updates `radius`, and returns `REF THIS`.
- Method `Draw()`: Prints circle specifications.

Student Tasks

1. Write the structured pseudo-code for `Point2D` and `Circle`.

2. Trace the execution of `myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()`.

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

Topic: Multilevel Inheritance, Constructors in Derived Classes, Base Pointers

Scenario

Design a multi-level inheritance hierarchy: `Vehicle` $\rightarrow$ `AutonomousVehicle` $\rightarrow$ `DeliveryDrone`.

- `Vehicle`: Carries `vehicleVIN` and `baseWeightKg`.
- `AutonomousVehicle`: Inherits `Vehicle`, adds `lidarSensorID` and `firmwareVersion`.
- `DeliveryDrone`: Inherits `AutonomousVehicle`, adds `payloadCapacityKg` and `batteryChargePercent`.

Functional Requirements

1. Constructor Delegation:

- Parameterized constructors must pass arguments through all three tiers using constructor initialization lists.

2. Telemetry Methods:

- `DisplaySpecs()` in `Vehicle`.
- `DisplayFlightStatus()` in `DeliveryDrone` (invokes `DisplaySpecs()` and prints drone-specific telemetry).

3. Pointer Management:

- Instantiate a `DeliveryDrone` dynamically on the heap.
- Assign its address to a `Vehicle*` base pointer to inspect general specs, then cast to derived pointer for full flight telemetry.

Student Tasks

1. Write the structured pseudo-code showing class definitions, constructor chains, and driver logic.

2. Trace constructor invocation order from root to leaf during instantiation.

Question 5: Dynamic Object Pointer Array & Zero-Copy Sorting Engine

Topic: Pointers to Objects, Arrow Operator (`->`), Pointer Manipulation

Scenario

Design an inventory sorting algorithm that operates on an **Array of Pointers to Objects** (`Product* catalog[N]`). Sorting must be achieved by swapping pointers rather than copying large product objects in memory.

Functional Requirements

1. Class `Product`:

- Fields: `sku` (STRING), `price` (REAL), `stockCount` (INTEGER).
- Parameterized constructor.
- Method `PrintProduct()`: Displays SKU, price, and stock count.

2. Sorting Logic:

- Accept an array of N dynamically allocated `Product*` pointers.
- Implement a pointer-swap Bubble Sort in ascending order of `price`.
- Crucial rule: Compare objects by dereferencing pointers via `catalog[j]->price`, but **swap only the pointer variables `catalog[j]` and `catalog[j+1]`**, keeping the actual objects stationary in memory.
- Deallocate all dynamic objects upon completion.

Student Tasks

1. Write the formal structured pseudo-code for `Product` and the pointer sorting driver.

2. Construct a dry-run trace table tracking pointer swaps for three products: Laptop (\$1200), Mouse (\$25), and Monitor (\$350).

SECTION A – DEBUGGING QUESTIONS

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

- Line 33 – Compilation error: request for member 'idNumber' is ambiguous. TeachingAssistant contains two Person subobjects: one inherited through Student and another through Employee.
- Non-virtual memory layout: TeachingAssistant contains Student → Person(idNumber = sId) and Employee → Person(idNumber = eId). The two paths create two separate base subobjects, so ta.idNumber does not identify a unique field.
- Correction: Student and Employee must inherit Person virtually. The most-derived class, TeachingAssistant, is responsible for initializing the shared virtual Person base.

```
#include <iostream>

class Person {
public:
    int idNumber;

    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: "
                  << idNumber << std::endl;
    }
};

class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};

class TeachingAssistant : public Student, public Employee {
public:
    TeachingAssistant(int id)
        : Person(id), Student(id), Employee(id) {}
};

int main() {
    TeachingAssistant ta(101);
    std::cout << "TA ID: " << ta.idNumber << std::endl;
    return 0;
}
```

Constructor rule: when a virtual base exists, only the most-derived class's initializer controls its construction. Person(101) in TeachingAssistant initializes the single shared Person subobject. Person initializers in Student and Employee are ignored when those classes are constructed as base subobjects of TeachingAssistant.

```
Person initialized: 101
TA ID: 101
```

Question 2: Missing Base Constructor in Derived Initializer List

- Line 18 – Compilation error: Device has only Device(std::string) and no default constructor. Since Sensor does not explicitly initialize Device, the compiler attempts to call Device(), which does not exist.
- The constructor body cannot assign macAddress in time to fix this: all base-class subobjects are constructed before the derived-class constructor body starts.
- Correct approach: invoke Device(mac) in Sensor's member initializer list. The chronological order is base constructor first, then Sensor data members, then the Sensor constructor body.

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;

public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: "
                  << macAddress << std::endl;
    }
};

class Sensor : public Device {
private:
    int samplingRate;

public:
    Sensor(std::string mac, int rate)
        : Device(mac), samplingRate(rate) {}

    void display() const {
        std::cout << "MAC: " << macAddress
                  << ", Rate: " << samplingRate
                  << " Hz" << std::endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Order	Construction step	Reason
1	Device(mac)	Base class is constructed first.
2	samplingRate(rate)	Derived class member is initialized.
3	Sensor constructor body	Runs after base and members are ready.
4	display()	Uses initialized base and derived fields.

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

Question 3: Base Class Pointer Member Access and Object Slicing

- Line 22 – Compilation error: vPtr has static type Vehicle*. Member access is checked against the pointer's static type, so only Vehicle members are directly available. The fact that the pointed-to object is a SportsCar does not make engageTurbo() a member of Vehicle.
- Line 26 – Object slicing: copying SportsCar into a Vehicle object copies only the Vehicle base subobject. The derived-only turboEnabled field is not part of plainVehicle.
- Use a checked downcast (dynamic_cast) with a polymorphic base, or a static_cast only when the programmer can guarantee the actual object type. Prevent slicing by using references or pointers instead of copying into a base object.

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;

    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() = default;

    void honk() const {
        std::cout << "General Horn" << std::endl;
    }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;

    SportsCar(int s, bool turbo)
        : Vehicle(s), turboEnabled(turbo) {}

    void engageTurbo() const {
        std::cout << "Turbo Engaged!" << std::endl;
    }
};

int main() {
    Vehicle* vPtr = new SportsCar(300, true);
    vPtr->honk();

    if (SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr)) {
        sportsPtr->engageTurbo();
    }

    SportsCar car(250, true);
    Vehicle& vehicleView = car; // No slicing
    std::cout << "Max speed: " << vehicleView.maxSpeed << std::endl;

    delete vPtr;
    return 0;
}
```

Dry run: vPtr points to a SportsCar object. honk() is accessible through Vehicle*. dynamic_cast checks the runtime object type and returns a SportsCar* on success. vehicleView aliases the Vehicle base part of car without creating a sliced Vehicle copy.

```
General Horn
Turbo Engaged!
Max speed: 250
```

Question 4: Incorrect Return Type in this Pointer Method Chaining

- Line 30 – Compilation error: setWidth returns WindowBox*. Therefore box.setWidth(800) is a pointer expression, and the following dot operator .setHeight(600) is invalid. A pointer would require ->, but the intended fluent syntax returns an object reference.
- this is a pointer to the current object (WindowBox* in this class). *this is the object itself as an lvalue expression; returning WindowBox& with return *this allows dot-based chaining.

```
#include <iostream>

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width
                  << " x " << height << std::endl;
    }
};

int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
    return 0;
}
```

```
Window: 800 x 600
```

Question 5: Private Inheritance Access Violations and Inaccessible Base Cast

- Line 24 – Access error: public and protected members of RawSocket become private members of SecureChannel under private inheritance. Client code in main() cannot call channel.closePort().

- Line 27 – Inaccessible-base error: the public conversion from SecureChannel* to RawSocket* is not accessible to outside code because RawSocket is a private base.
- Approach A: publish the required base operation using a public using-declaration. This makes closePort() callable through SecureChannel's public interface, but it does not make the RawSocket base conversion public.
- Approach B: change to public inheritance if SecureChannel is-a RawSocket and public substitutability is intended; alternatively use composition and explicitly forward selected operations.

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened." << std::endl;
    }

    void closePort() {
        std::cout << "Port closed." << std::endl;
    }
};

class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;

    void establishEncryptedTunnel(int port) {
        openPort(port);
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();
    return 0;
}
```

The corrected code uses the requested Approach A. It intentionally does not perform RawSocket* socketPtr = &channel because private inheritance still makes that external upcast inaccessible.

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

- Person is the shared virtual root. Student and Faculty virtually inherit Person, and TeachingAssistant inherits both. This prevents duplicate Person records.
- TeachingAssistant, as the most-derived class, directly initializes the virtual Person base.

```
CLASS Person
    PRIVATE:
        personID: STRING
        fullName: STRING
        email: STRING

    PUBLIC:
        CONSTRUCTOR Person(id: STRING, name: STRING, mail: STRING)
            personID <- id
            fullName <- name
            email <- mail
        END

        METHOD DisplayPerson()
            OUTPUT personID, " ", fullName, " ", email
        END
ENDCLASS

CLASS Student EXTENDS VIRTUAL Person
    PRIVATE:
        gpa: REAL
        creditsEarned: INTEGER

    PUBLIC:
        CONSTRUCTOR Student(id, name, mail, grade, credits)
            : BASE Person(id, name, mail)
            gpa <- grade
            creditsEarned <- credits
        END
ENDCLASS

CLASS Faculty EXTENDS VIRTUAL Person
    PRIVATE:
        academicDepartment: STRING
        rankTitle: STRING

    PUBLIC:
        CONSTRUCTOR Faculty(id, name, mail, dept, rank)
            : BASE Person(id, name, mail)
            academicDepartment <- dept
            rankTitle <- rank
        END
ENDCLASS

CLASS TeachingAssistant EXTENDS Student, Faculty
    PRIVATE:
        stipendAmount: REAL
        assignedCourse: STRING
```

```
    PUBLIC:
        CONSTRUCTOR TeachingAssistant(id, name, mail, grade, credits,
                                      dept, rank, stipend, course)
            : BASE Person(id, name, mail)
            : Student(id, name, mail, grade, credits)
            : Faculty(id, name, mail, dept, rank)
            stipendAmount <- stipend
            assignedCourse <- course
        END

        METHOD DisplayTARecord()
            OUTPUT "Person ID: ", personID
            OUTPUT "Name: ", fullName
            OUTPUT "GPA: ", gpa
            OUTPUT "Department: ", academicDepartment
            OUTPUT "Stipend: ", stipendAmount
            OUTPUT "Assigned Course: ", assignedCourse
        END
ENDCLASS

BEGIN Main
    DECLARE ta: TeachingAssistant
    ta <- NEW TeachingAssistant("P102", "Kavin",
         "kavin@university.edu", 8.7, 92,
         "Computer Science", "Assistant Professor",
         12000, "OOP with C++")
    CALL ta.DisplayTARecord()
END
```

Field	Sample value
Person ID	P102
Name	Kavin
GPA	8.7
Department	Computer Science
Stipend	12,000
Assigned Course	OOP with C++

```
Person ID: P102
Name: Kavin
GPA: 8.7
Department: Computer Science
Stipend: 12000
Assigned Course: OOP with C++
```

Question 2: Polymorphic Payroll with Abstract Base Class and Base Pointers

- Employee is abstract because CalculatePay() is a pure virtual method. Employee* pointers can point to concrete derived objects and invoke the appropriate override polymorphically.
- The pointer array is dynamically allocated. Each Employee object is also dynamically allocated, and all objects plus the pointer array must be released. In C++, the base class should have a virtual destructor when deleting derived objects through Employee*.

```
ABSTRACT CLASS Employee
    PROTECTED:
        employeeID: STRING
        name: STRING

    PUBLIC:
```

```
        CONSTRUCTOR Employee(id: STRING, empName: STRING)
            employeeID <- id
            name <- empName
        END

        ABSTRACT METHOD CalculatePay() RETURNS REAL

        METHOD DisplayHeader()
            OUTPUT employeeID, " ", name
        END

        VIRTUAL DESTRUCTOR()
            // Release base-owned resources, if any.
        END
ENDCLASS

CLASS SalariedEmployee EXTENDS Employee
    PRIVATE:
        monthlySalary: REAL

    PUBLIC:
        CONSTRUCTOR SalariedEmployee(id, name, salary)
            : BASE Employee(id, name)
            monthlySalary <- salary
        END

        METHOD CalculatePay() RETURNS REAL
            RETURN monthlySalary
        END
ENDCLASS

CLASS HourlyEmployee EXTENDS Employee
    PRIVATE:
        hourlyWage: REAL
        hoursWorked: REAL

    PUBLIC:
        CONSTRUCTOR HourlyEmployee(id, name, wage, hours)
            : BASE Employee(id, name)
            hourlyWage <- wage
            hoursWorked <- hours
        END

        METHOD CalculatePay() RETURNS REAL
            IF hoursWorked <= 40 THEN
                RETURN hourlyWage * hoursWorked
            ELSE
                RETURN (hourlyWage * 40) +
                       (hourlyWage * 1.5 * (hoursWorked - 40))
            ENDIF
        END
ENDCLASS

BEGIN Main
    DECLARE employees AS ARRAY OF POINTER TO Employee
    ALLOCATE ARRAY[2] OF POINTER TO Employee
```

```
    employees[0] <- NEW SalariedEmployee("E101", "Mira", 8500)
    employees[1] <- NEW HourlyEmployee("E102", "Ravi", 50, 48)

    DECLARE totalPay: REAL <- 0
    FOR i <- 0 TO 1
        employees[i]->DisplayHeader()
        DECLARE pay: REAL
        pay <- employees[i]->CalculatePay()
        OUTPUT "Pay: ", pay
        totalPay <- totalPay + pay
    ENDFOR

    OUTPUT "Total disbursement: ", totalPay

    FOR i <- 0 TO 1
        DELETE employees[i]
    ENDFOR
    DEALLOCATE employees
END
```

Employee	Calculation	Pay
Salaried	Fixed monthly salary	8,500
Hourly regular	50 × 40	2,000
Hourly overtime	50 × 1.5 × 8	600
Hourly total	2,000 + 600	2,600
Total disbursement	8,500 + 2,600	11,100

Total payroll disbursement = 11,100.

Question 3: Composite Canvas Engine with Member Classes and this Chaining

- Circle has a Point2D member object representing its center. This is composition (has-a), not inheritance (is-a).
- SetCenter and SetRadius return REF THIS so the next method in the chain is invoked on the same Circle object.

```
CLASS Point2D
    PRIVATE:
        x: REAL
        y: REAL

    PUBLIC:
        CONSTRUCTOR Point2D(a: REAL = 0.0, b: REAL = 0.0)
            x <- a
            y <- b
        END

        METHOD SetCoordinates(a: REAL, b: REAL)
            x <- a
            y <- b
        END

        METHOD Display()
            OUTPUT "(", x, ", ", y, ")"
        END
```

```
ENDCLASS

CLASS Circle
    PRIVATE:
        centerPoint: Point2D
        radius: REAL

    PUBLIC:
        CONSTRUCTOR Circle()
            centerPoint <- Point2D(0.0, 0.0)
            radius <- 1.0
        END

        METHOD SetCenter(newX: REAL, newY: REAL)
            CALL centerPoint.SetCoordinates(newX, newY)
            RETURN REF THIS
        END

        METHOD SetRadius(r: REAL)
            IF r > 0 THEN
                radius <- r
            ELSE
                OUTPUT "Invalid radius"
            ENDIF
            RETURN REF THIS
        END

        METHOD Draw()
            OUTPUT "Circle center: "
            CALL centerPoint.Display()
            OUTPUT "Radius: ", radius
        END
ENDCLASS

BEGIN Main
    DECLARE myCircle: Circle
    CALL myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()
END
```

Call	Effect	Object state
Initial constructor	Origin and default radius	Center (0.0, 0.0), radius 1.0
SetCenter(12.5, 30.0)	Updates Point2D member	Center (12.5, 30.0), radius 1.0
SetRadius(7.5)	Positive radius accepted	Center (12.5, 30.0), radius 7.5
Draw()	Displays current state	Center (12.5, 30.0), radius 7.5

```
Circle center: (12.5, 30.0)
Radius: 7.5
```

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

- The inheritance chain is Vehicle → AutonomousVehicle → DeliveryDrone. Each constructor passes its base-class arguments through the member initializer list.
- Construction order is root to leaf: Vehicle, AutonomousVehicle, then DeliveryDrone. The driver creates the drone dynamically, uses a Vehicle* for general vehicle details, then safely converts to DeliveryDrone* for drone-specific telemetry.

```
CLASS Vehicle
    PROTECTED:
        vehicleVIN: STRING
        baseWeightKg: REAL

    PUBLIC:
        CONSTRUCTOR Vehicle(vin: STRING, weight: REAL)
            vehicleVIN <- vin
            baseWeightKg <- weight
        END

        METHOD DisplaySpecs()
            OUTPUT "VIN: ", vehicleVIN
            OUTPUT "Base weight: ", baseWeightKg, " kg"
        END

        VIRTUAL DESTRUCTOR()
        END
ENDCLASS

CLASS AutonomousVehicle EXTENDS Vehicle
    PROTECTED:
        lidarSensorID: STRING
        firmwareVersion: STRING

    PUBLIC:
        CONSTRUCTOR AutonomousVehicle(vin, weight, lidar, firmware)
            : BASE Vehicle(vin, weight)
            lidarSensorID <- lidar
            firmwareVersion <- firmware
        END
ENDCLASS

CLASS DeliveryDrone EXTENDS AutonomousVehicle
    PRIVATE:
        payloadCapacityKg: REAL
        batteryChargePercent: REAL

    PUBLIC:
        CONSTRUCTOR DeliveryDrone(vin, weight, lidar, firmware,
                                  payload, battery)
            : BASE AutonomousVehicle(vin, weight, lidar, firmware)
            payloadCapacityKg <- payload
            batteryChargePercent <- battery
        END

        METHOD DisplayFlightStatus()
            CALL DisplaySpecs()
            OUTPUT "LiDAR ID: ", lidarSensorID
            OUTPUT "Firmware: ", firmwareVersion
            OUTPUT "Payload capacity: ", payloadCapacityKg, " kg"
            OUTPUT "Battery: ", batteryChargePercent, "%"
        END
ENDCLASS

BEGIN Main
    DECLARE vehiclePtr AS POINTER TO Vehicle
```

```
    DECLARE dronePtr AS POINTER TO DeliveryDrone

    vehiclePtr <- NEW DeliveryDrone("VIN-D100", 12.5,
                  "LIDAR-77", "v2.4.1", 4.0, 88.0)

    CALL vehiclePtr->DisplaySpecs()

    dronePtr <- DYNAMIC_CAST(vehiclePtr AS POINTER TO DeliveryDrone)
    IF dronePtr != NULL THEN
        CALL dronePtr->DisplayFlightStatus()
    ENDIF

    DELETE vehiclePtr
END
```

Construction order	Object component	Values initialized
1	Vehicle	VIN-D100; 12.5 kg
2	AutonomousVehicle	LIDAR-77; firmware v2.4.1
3	DeliveryDrone	Payload 4.0 kg; battery 88.0%

The call through Vehicle* can access Vehicle-declared members. The derived cast is required for DeliveryDrone-specific flight telemetry. A virtual destructor ensures correct destruction through the base pointer.

```
VIN: VIN-D100
Base weight: 12.5 kg
VIN: VIN-D100
Base weight: 12.5 kg
LiDAR ID: LIDAR-77
Firmware: v2.4.1
Payload capacity: 4 kg
Battery: 88%
```

Question 5: Dynamic Object Pointer Array and Zero-Copy Sorting Engine

- The catalog is an array of Product* pointers. The comparison reads each pointed-to object's price using ->, but the sort swaps only pointer variables. Product objects are not copied or moved during sorting.
- For three products, the initial pointer order is Laptop ($1,200), Mouse ($25), Monitor ($350). Ascending bubble sort performs two pointer swaps and results in Mouse, Monitor, Laptop.

```
CLASS Product
    PRIVATE:
        sku: STRING
        price: REAL
        stockCount: INTEGER

    PUBLIC:
        CONSTRUCTOR Product(code: STRING, cost: REAL, stock: INTEGER)
            sku <- code
            price <- cost
            stockCount <- stock
        END

        METHOD PrintProduct()
            OUTPUT sku, " | ", price, " | ", stockCount
```

```
        END
ENDCLASS

BEGIN Main
    DECLARE N: INTEGER <- 3
    DECLARE catalog AS ARRAY[N] OF POINTER TO Product

    catalog[0] <- NEW Product("Laptop", 1200, 8)
    catalog[1] <- NEW Product("Mouse", 25, 40)
    catalog[2] <- NEW Product("Monitor", 350, 12)

    // Bubble sort ascending by price; swap pointer values only.
    FOR pass <- 0 TO N - 2
        FOR j <- 0 TO N - 2 - pass
            IF catalog[j]->price > catalog[j + 1]->price THEN
                DECLARE temp AS POINTER TO Product
                temp <- catalog[j]
                catalog[j] <- catalog[j + 1]
                catalog[j + 1] <- temp
            ENDIF
        ENDFOR
    ENDFOR

    OUTPUT "Sorted catalog:"
    FOR i <- 0 TO N - 1
        CALL catalog[i]->PrintProduct()
    ENDFOR

    FOR i <- 0 TO N - 1
        DELETE catalog[i]
        catalog[i] <- NULL
    ENDFOR
END
```

Pass / comparison	Compared prices	Pointer action	Pointer order after comparison
Initial	—	—	Laptop, Mouse, Monitor
Pass 1, j=0	1200 > 25	Swap catalog[0], catalog[1]	Mouse, Laptop, Monitor
Pass 1, j=1	1200 > 350	Swap catalog[1], catalog[2]	Mouse, Monitor, Laptop
Pass 2, j=0	25 > 350 is false	No swap	Mouse, Monitor, Laptop

Sorted output:

```
Mouse | 25 | 40
Monitor | 350 | 12
Laptop | 1200 | 8
```

Time complexity: $O(N^2)$ for bubble sort. Auxiliary sorting space: O(1), because only one temporary pointer is used. The dynamically allocated Product objects remain at their original memory locations; only the pointer order changes. Every object is deleted exactly once after sorting.